DataFrameLib

A High-Performance DataFrame Library in C++

COP290 - Design Practices | Assignment 4

Project Report

Index

1. File Structure 2

2. Design and Design Decisions 2

3. EagerDataFrame - Implementation 3

3.1 Working. 3

3.2 Related Files and Code. 4

4. LazyDataFrame - Implementation 5

4.1 Working. 5

4.2 Related Files and Code. 6

5. QueryOptimiser - Implementation 7

5.1 Working. 7

5.2 Optimization Rules - Detailed Analysis 7

5.3 When Optimization Can Hurt Performance 9

6. Supporting Components 10

6.1 Apache Arrow Layer 10

6.2 Graph Module 11

7. Allowed Data Types 11

8. Using DataFrameLib in Other Projects 11

9. Input Format and Correct Usage 12

10. Exception Handling 13

1. File Structure

The project is organised into structure that separates header files (declarations) from source files (implementations). All public headers are placed under a single include directory to keep the public API together, and all source files are placed under src. The structure is shown below:

```
project/
|-- include/
|   +-- dataframelib/
|       |-- apachearrow.h
|       |-- dataframelib.h
|       |-- eager_helper.h
|       |-- eagerdataframe.h
|       |-- expression.h
|       |-- graph.h
|       |-- lazydataframe.h
|       +-- queryoptimiser.h
|-- src/
|   |-- apachearrow.cpp
|   |-- eager_helper.cpp
|   |-- eagerdataframe.cpp
|   |-- expr.cpp
|   |-- expression.cpp
|   |-- graph.cpp
|   |-- lazydataframe.cpp
|   +-- queryoptimiser.cpp
|-- tests/
|-- CMakeLists.txt
+-- README
```

A short description of each file:

- **dataframelib.h:** the master include file. A user only needs to include this single header to get access to the entire library.
- **expression.h / expression.cpp / expr.cpp:** define the expression system (columns, literals, arithmetic, comparisons, string functions, aggregations) used inside filters, projections, and aggregations.
- **eagerdataframe.h / eagerdataframe.cpp:** define the EagerDataFrame class along with GroupedDataFrame for eager (immediate) execution.
- **eager_helper.h / eager_helper.cpp:** helper routines used by the eager path - join key building, null-safe row taking, group-by helpers, aggregation evaluation, and final table assembly.
- **lazydataframe.h / lazydataframe.cpp:** define LazyDataFrame which builds a DAG of operations and only executes when collect() is called.
- **graph.h / graph.cpp:** the Node structure used as building blocks of the lazy plan, plus utilities to print and export the plan to a Graphviz .dot / .png file.
- **queryoptimiser.h / queryoptimiser.cpp:** the rule-based query optimiser - it walks the plan tree and applies transformations like predicate pushdown, projection pruning, and expression simplification.
- **apachearrow.h / apachearrow.cpp:** thin layer of utility functions that wrap raw Apache Arrow operations (mainly manual_take_table for selecting rows by index).
- **CMakeLists.txt:** single build file at the project root which compiles the whole library and links Apache Arrow / Parquet.

2. Design and Design Decisions

2.1 Apache Arrow as the storage layer

Every DataFrame stores its data in a std::shared_ptr<arrow::Table>. Apache Arrow provides a column-oriented, cache-friendly memory layout, native handling of null values, and built-in CSV / Parquet readers and writers. This means our library inherits high-performance I/O for free instead of writing custom parsers.

2.2 Two execution modes

We follow the same model as Polars - operations can be performed in two distinct modes:

- **Eager mode:** every method call (filter, select, sort, ...) runs immediately and returns a new EagerDataFrame holding the materialised result. This is best for interactive or small-data work.
- **Lazy mode:** method calls only build a tree of Node objects. No data is touched until collect() is invoked. This makes optimisation possible because we can rewrite the tree before execution.

2.3 Expression system as a separate tree

Inside a single operation, expressions like col("a") + col("b") > 10 are themselves represented as a tree of Expression objects. Each expression has its own evaluate() function. This isolates expression logic from DataFrame logic and lets us reuse the same expressions in eager and lazy paths.

2.4 Smart pointers everywhere

We use std::shared_ptr for all heap-allocated objects (Arrow tables, expressions, plan nodes). This removes manual memory management and makes the library safe against leaks.

2.5 API

Every transforming operation returns a new DataFrame (or LazyDataFrame), so users can chain calls naturally:

```
df.filter(col("age") > 30)
  .select({"name", "salary"})
  .sort({"salary"}, false);
```

2.6 Helper file separation

Long helper functions (build_groups, evaluate_aggregations, take_with_nulls, etc.) are placed in eager_helper.cpp instead of bloating the main eagerdataframe.cpp. This keeps each file focused on one job and easier to read.

3. EagerDataFrame - Implementation

3.1 Working

EagerDataFrame is the immediate-execution DataFrame. Internally it stores a single private member:

```
std::shared_ptr<arrow::Table> table_;
```

Every operation reads from this table and produces a new EagerDataFrame holding a new table. The flow of a typical operation is:

- the operation receives a column list or an expression,
- it reads the relevant columns from table_,
- it builds new Arrow arrays / chunked arrays using Arrow builders,
- it assembles a new arrow::Table and wraps it in a new EagerDataFrame.

Each operation in our code follows this pattern. For example,

```
EagerDataFrame EagerDataFrame::select(
    const std::vector<std::string>& colnames) {
    std::vector<std::shared_ptr<arrow::Field>> fields;
    std::vector<std::shared_ptr<arrow::ChunkedArray>> cols;
    for (const auto& name : colnames) {
        auto col = table_->GetColumnByName(name);
        if (!col) throw std::invalid_argument(
            "select: column not found: " + name);
```

```
        fields.push_back(table_->schema()->GetFieldByName(name));
        cols.push_back(col);
    }
    auto schema = std::make_shared<arrow::Schema>(fields);
    return EagerDataFrame(arrow::Table::Make(schema, cols));
}
```

filter() evaluates the predicate expression against the table, walks every row, and collects indices of rows where the result is true. It then uses manual_take_table from the Arrow helper to physically build the filtered table.

sort() builds a vector of indices [0..n), sorts these indices using a custom comparator that handles strings, numerics, and nulls (nulls last), and finally takes the rows in that order.

join() builds a hash map from the right table's join key to its row indices, then walks the left table, looks up matches, and stores pairs of indices (L, R). It supports inner, left, right, and outer joins. For inner joins it uses manual_take_table; for the others it uses take_with_nulls so that unmatched rows correctly get nulls. Finally it coalesces join-key columns and renames duplicate columns by appending _right.

group_by() doesn't compute anything itself; it returns a GroupedDataFrame holding the table and the key list. The actual work happens in aggregate(), which uses helpers to:

- build groups (rows grouped by the key tuple),
- evaluate each aggregation expression on each group,
- build the final key-table + aggregation-columns table.

3.2 Related Files and Code

3.2.1 eagerdataframe.h

Declares the EagerDataFrame and GroupedDataFrame classes, plus free functions read_csv, read_parquet, and from_columns (overloaded for std::map, std::unordered_map, std::vector<pair>, and initializer_list).

3.2.2 eagerdataframe.cpp

Contains the implementation of every public EagerDataFrame method. Notable points:

- read_csv / read_parquet open the file with arrow::io::ReadableFile, build the appropriate reader, and return an EagerDataFrame wrapping the resulting table.
- write_csv / write_parquet open an output stream and serialise the table using Arrow's built-in writers.
- with_column checks whether the column already exists - if it does, it replaces it; if not, it appends a new column. This matches Polars semantics.
- head simply uses table_->Slice(0, n) which is a zero-copy slice.

3.2.3 eager_helper.h / eager_helper.cpp

Contains the reusable building blocks used by joins and group-by:

- **get_bool_value_safe:** safely extracts a bool from an Arrow scalar. Returns false on null or non-bool, which keeps filter() safe against bad expressions.
- **make_key:** concatenates the values of the join columns into a single string used as a hash-map key.
- **take_with_nulls:** like manual_take_table but supports negative indices (-1) which it materialises as null - needed for left/right/outer joins.
- **append_scalar_to_builder:** appends a typed Arrow scalar to the right kind of builder (Int64Builder, DoubleBuilder, StringBuilder, ...). Centralises type dispatch in one place.
- **build_groups:** scans the table once and groups row indices by their key string.
- **evaluate_aggregations:** for each group, slices the table to that group's rows and evaluates each aggregation expression on the slice.
- **build_final_table:** concatenates the key columns and aggregation columns into the final result table.

3.2.4 expression.h, expression.cpp, expr.cpp

Define the Expression hierarchy used inside filter / with_column / aggregate:

- ColumnExpression: returns the column as an Arrow Datum.
- LiteralExpression: holds a std::variant<int64_t, double, std::string, bool>.
- BinaryExpression: handles +, -, *, /, %, comparisons, &, |.
- UnaryExpression: handles ~, is_null, is_not_null.
- FunctionExpression: handles length, contains, starts_with, ends_with, to_lower, to_upper, abs.
- AggregationExpression: handles sum, mean, count, min, max.
- AliasExpression: renames the result of another expression.

expr.cpp wraps these into a friendly Expr class with overloaded operators (+, -, *, /, ==, !=, <, >, &, |, ~) so users can write col("x") * 2 + col("y") naturally instead of constructing tree nodes by hand.

4. LazyDataFrame - Implementation

4.1 Working

LazyDataFrame stores only one thing - the root of a query plan tree:

```
std::shared_ptr<Node> plan_root_;
```

When the user calls scan_csv(path) or scan_parquet(path), we create a leaf Node with op = "scan_csv" / "scan_parquet" and store the file path inside it. Every subsequent transformation (filter, select, with_column, sort, head, join, group_by, aggregate) creates a new Node whose op tag identifies the operation, copies the relevant data into the node, and adds the previous plan as its child:

```
LazyDataFrame LazyDataFrame::filter(
    const std::shared_ptr<Expression>& expr) {
    auto node = std::make_shared<Node>("filter");
    node->children.push_back(plan_root_);
    node->expr = expr;
    return LazyDataFrame(node);
}
```

No data is read or processed at this point - we are only building a tree. Execution is triggered by collect():

```
EagerDataFrame LazyDataFrame::collect() {
    plan_root_ = optimize(plan_root_);
    return execute(plan_root_);
}
```

The plan first goes through the QueryOptimiser, then through the execute() function which walks the tree bottom-up: leaf nodes (scans) read the file into an EagerDataFrame, and every other node calls the matching EagerDataFrame method on the result of its child.

Special handling exists for join (it has both children[0] and right_child) and for the group_by + aggregate pair (aggregate's child must be a group_by node, otherwise we throw an error).

Apart from collect(), LazyDataFrame also exposes:

- **explain(path):** writes the optimised DAG as a Graphviz .dot file and converts it to .png using the system dot command. This satisfies the assignment's plan-visualisation requirement.
- **sink_csv / sink_parquet:** shortcut for collect().write_csv(...) / write_parquet(...) so the user can stream the result straight to disk in one call.

4.2 Related Files and Code

4.2.1 lazydataframe.h

Declares the LazyDataFrame class. Every operation method is small - it only records the operation. The header also declares free helpers scan_csv(path) and scan_parquet(path) which simply forward to the static class methods.

4.2.2 lazydataframe.cpp

Contains all operation builders plus the execute() function. execute() is the core lazy-side dispatcher:

```
if (node->op == "scan_csv")
    return EagerDataFrame::read_csv(node->file_path);
else if (node->op == "scan_parquet")
    return EagerDataFrame::read_parquet(node->file_path);

auto input = execute(node->children[0]);

if (node->op == "filter")
    return input.filter(node->expr);
else if (node->op == "select")
    return input.select(node->columns);
else if (node->op == "join") {
    auto right = execute(node->right_child);
    return input.join(right, node->columns, node->join_type);
}
// ... more cases ...
```

The aggregate case is slightly different: it expects its child to be a group_by node, so we read the underlying table from group_by's child and call data.group_by(keys).aggregate(map).

4.2.3 graph.h / graph.cpp

Defines the Node struct, which is the universal building block of every plan. Node carries everything any operation might need:

```
class Node {
public:
    std::string op;                                     // "filter", "select", ...
    std::vector<std::shared_ptr<Node>> children;        // one child for unary ops
    std::shared_ptr<Node> right_child;                  // second input for join
    std::shared_ptr<Expression> expr;                   // for filter / with_column
    std::vector<std::string> columns;                   // for select / sort / group_by
    std::string file_path;                              // for scan_csv / scan_parquet
    std::map<std::string,std::shared_ptr<Expression>> agg_map;
    std::string join_type;
    bool ascending = true;
    int head_n = 0;
};
```

Using one universal Node type (instead of a separate class per operation) keeps the optimiser code simple - we only need to inspect the op string.

graph.cpp also implements:

- **print_plan:** prints the plan to standard output with indentation, useful for debugging.
- **export_to_dot:** assigns a unique id to every node and writes a Graphviz .dot file that LazyDataFrame::explain() then converts to PNG.

5. QueryOptimiser - Implementation

5.1 Working

The optimiser is a single recursive function optimize(node, depth) that walks the plan tree bottom-up. Its job is to rewrite the tree into a logically-equivalent but cheaper one before execute() runs. The structure is:

- recurse into all children first (depth-first, post-order),
- simplify any expression attached to the current node,

- apply pattern-matching rules between the current node and its child.

A safety guard MAX_STEPS = 200 prevents infinite recursion in case a rule unexpectedly creates a cycle. The depth parameter is incremented on every call.

```
std::shared_ptr<Node> optimize(std::shared_ptr<Node> node, int depth = 0) {
    if (!node) return node;
    if (depth > 200) return node;            // safety guard

    for (auto& child : node->children)
        child = optimize(child, depth + 1);
    if (node->right_child)
        node->right_child = optimize(node->right_child, depth + 1);

    if (node->expr)
        node->expr = simplify(node->expr);

    // ... pattern-matching rules ...
    return node;
}
```

5.2 Optimization Rules Analysis

The optimiser implements all five standard rules listed in the assignment.

5.2.1 Constant Folding and Expression Simplification

Transformation: The simplify(expr) function walks an expression tree bottom-up. When both children of a binary operator are literal values (constants), it evaluates the operation at plan-construction time and replaces the entire subtree with a single LiteralExpression. Additionally, it applies algebraic identities: x*1 -> x, x+0 -> x, x*0 -> 0, true & x -> x, false | x -> false, etc.

Correctness Proof: These are standard mathematical identities. Replacing lit(a) op lit(b) with lit(result) is valid because literals are independent of any row data - their value is the same regardless of the table contents. The algebraic identities (additive identity, multiplicative identity, multiplicative annihilator) and boolean identities (AND-identity, OR-identity) hold universally.

Example: Consider the expression col("a") + 0 > 1 * 5. The simplifier first processes 1 * 5 into lit(5), then processes col("a") + 0 into col("a"), and the final result is col("a") > 5. The plan goes from evaluating three operations per row (add, multiply, compare) to evaluating just one (compare).

Performance Benefit: Eliminates redundant per-row computations. In a table with N rows, each folded-away operation saves N evaluations. If constant sub-expressions appear inside a tight filter on a million-row dataset, the savings are substantial.

5.2.2 Predicate Pushdown through Join

Transformation: When a filter node sits directly above a join node and all columns referenced by the filter predicate belong to only one side of the join (left or right), the filter is detached from above the join and attached below it, directly on the relevant input side.

```
// Before optimization:    filter -> join(left, right)
// After optimization:     join(filter -> left, right)

if (node->op == "filter" && child->op == "join") {
    auto filter_cols = extract_columns(node->expr);
    auto left_cols  = check_parent_columns(left_child);
    auto right_cols = check_parent_columns(right_child);
    if (contains_all(left_cols, filter_cols))
        // push filter to left side
}
```

Correctness Proof: If a filter references only left-side columns, then a row either passes the filter or it does not, regardless of what it is paired with from the right side. Filtering the left input first produces a smaller left table; the subsequent join then pairs this smaller set with the full right table. The final output is identical because no row that would have survived both the join and the post-join filter is removed, and no extra row is introduced.

Example: Suppose we have a 100,000-row employees table joined with a 50-row departments table on dept_id, followed by filter(col("salary") > 80000). Since salary belongs to employees, the filter is pushed before the join. If only 10% of employees pass the filter, the join now processes 10,000 rows instead of 100,000 - a 10x reduction in join work.

Performance Benefit: Joins are typically O(N*M) or O(N+M) with hashing. Reducing N (or M) before the join has a direct, often dramatic, impact on total execution time. This is widely regarded as the single most impactful optimisation in query planning.

5.2.3 Predicate Pushdown through Select, With_Column, and Sort

Transformation: Three additional pushdown rules move filter nodes past other operations:

- filter -> select: if every column the filter references is retained by the select, the filter is moved below the select. The select then operates on fewer rows.
- filter -> with_column: if the filter does not reference the newly created column, the filter is moved below the with_column. The new column is then computed on fewer rows.
- filter -> sort: the filter is unconditionally moved below the sort, because sorting fewer rows is always cheaper.

Correctness Proof: In each case the filter's predicate depends only on columns that exist before the operation above it. Moving the filter earlier does not change which rows survive - it merely changes the order of independent operations. Specifically: select only drops columns (never rows), so filtering before or after produces the same surviving rows. with_column adds a new column but does not change existing data, so a filter not referencing the new column sees identical inputs either way. sort reorders rows but does not add or remove any, so filter and sort commute freely.

Example: Consider scan_csv -> sort({"name"}, true) -> filter(col("age") > 30). The filter is pushed below the sort, yielding scan_csv -> filter(col("age") > 30) -> sort({"name"}, true). If the table has 50,000 rows and the filter keeps 5,000, the sort now processes 5,000 rows instead of 50,000. Since sort is O(N log N), this reduces sort work by roughly 10x.

Performance Benefit: Reduces the row count seen by downstream, potentially expensive operations like sort (O(N log N)) and with_column (O(N) per new column). The earlier the filter runs, the fewer rows flow through the rest of the pipeline.

5.2.4 Projection Pushdown

Transformation: Two rules reduce the number of columns processed:

- select -> select: two consecutive selects are collapsed into one whose column list is the intersection of both. This avoids materialising an intermediate table with columns that will immediately be dropped.
- select -> with_column: if the select does not include the column created by with_column, the with_column node is bypassed entirely, because its output is never used.

Correctness Proof: For select-select: if the outer select keeps columns {A, B} and the inner select keeps {A, B, C}, the visible output is {A, B} regardless. Collapsing them keeps {A, B} directly. For select-with_column: if a with_column creates column Z but the downstream select never includes Z, then Z is computed but discarded. Removing the with_column entirely is safe because no downstream operation references Z.

Example: Consider scan_csv -> with_column("tax", col("price") * 0.18) -> select({"id", "price"}). Since the select does not include "tax", the with_column is removed. This skips the per-row multiplication entirely - saving N floating-point operations.

Performance Benefit: Avoids computing unused columns (saving CPU) and avoids materialising unused data (saving memory). In wide tables with many columns, projection pushdown can dramatically reduce the amount of data that flows through the pipeline.

5.2.5 Limit Pushdown

Transformation: Three rules push row-count limits earlier:

- head -> head: two consecutive heads are collapsed into one with the smaller of the two n values.
- head -> with_column: the head is pushed below the with_column, so the new column is only computed for the first n rows rather than for the entire table.
- select -> sort: if the select retains all sort-key columns, the select is pushed below the sort. Although this is not strictly a limit rule, it lets the sort operate on narrower rows (fewer columns), improving cache utilisation.

Correctness Proof: For head-head: head(n) followed by head(m) keeps min(n,m) rows. A single head(min(n,m)) produces the same result. For head-with_column: with_column does not change existing rows or their order; it only appends a new column. Applying head first and then computing the new column produces the same first-n rows with the same new-column values. For select-sort: select removes columns but does not affect row order or content of the kept columns, so the sort's result is the same whether columns are dropped before or after sorting.

Example: Consider scan_csv -> with_column("bonus", col("salary")*0.1) -> head(10) on a 100,000-row table. After optimisation the plan becomes scan_csv -> head(10) -> with_column("bonus", col("salary")*0.1). The bonus multiplication runs on 10 rows instead of 100,000.

Performance Benefit: Avoids processing the full dataset when only a small prefix is needed. On large tables where head(n) with small n is common (e.g. previewing data), this turns O(N) operations into O(n).

5.2.6 Helper Functions

The optimiser uses small helpers:

- **extract_columns(expr):** recursively walks an Expression tree and returns all column names it references. This determines whether a filter can be pushed to the left or right side of a join.
- **contains_all(parent, child):** set-containment check used to verify that all required columns exist in a given scope.
- **check_parent_columns(node):** for a leaf scan node it reads the actual column names from the CSV header or Parquet schema; for inner nodes it recurses. This gives accurate column information rather than guessing.

5.3 When Optimization Can Hurt Performance

While the optimiser generally improves execution time, there are specific scenarios where the optimised plan runs slower than the unoptimised one. Through testing, we identified three categories of overhead.

5.3.1 Small Query Overhead

For queries whose total execution time is very low (under 2ms), the optimizer's own cost can exceed the savings it produces. The optimize() function is a recursive process that traverses the entire plan tree, checks patterns, simplifies expressions, and reconstructs nodes. For trivial queries (e.g. a single filter or a small with_column), the time spent analysing the plan is longer than the time the optimisation would save.

5.3.2 File I/O during Optimization

The check_parent_columns function, opens the source CSV or Parquet file and reads its header to extract column names. Opening a file on disk - even just to read the header - is a heavy operation compared to in-memory work.

On small datasets where the entire read-filter-join pipeline takes only a few milliseconds, this disk-access overhead outweighs the execution benefit of predicate pushdown. The overhead is proportional to the number of scan nodes in the plan, not the data size.

A potential improvement would be to cache the column metadata after the first read, or to extract it from the plan's schema information rather than re-reading the file.

5.3.3 System Noise on Larger Queries

On larger tasks we observed occasional slowdowns of 5-10% . These variations are within the range of system noise - fluctuations caused by OS background tasks, CPU frequency scaling, cache state, and memory allocation patterns.

To distinguish genuine optimiser overhead from noise, benchmarks should be run multiple times and averaged.

6. Supporting Components

6.1 Apache Arrow Layer

apachearrow.h / apachearrow.cpp expose one critical helper:

```
std::shared_ptr<arrow::Table> manual_take_table(
    const std::shared_ptr<arrow::Table>& table,
    const std::vector<int64_t>& indices);
```

This function takes a list of row indices and produces a new table containing only those rows, in that order. It is the foundation of filter, sort, group_by, and inner-join. The implementation walks each column once, creates a typed Arrow builder (Int32Builder, Int64Builder, DoubleBuilder, StringBuilder, BooleanBuilder, ...) and appends the requested values one by one. Null cells are appended as nulls.

6.2 Graph Module

graph.h / graph.cpp provide the data structure and tooling for the lazy plan tree (already discussed in section 4.2.3). The two visualisation helpers it exports are:

- **print_plan(node, depth=0):** prints the plan tree to stdout with indentation. Each level adds two spaces, so deeply nested plans are still readable.
- **export_to_dot(root, filename):** writes the plan as a Graphviz .dot file. Internally it uses a depth-first traversal to assign each node a unique integer id, then emits node_id [label="op"]; lines and edge lines from each child to its parent.

LazyDataFrame::explain(path) glues these together: it writes a temporary .dot file, runs dot -Tpng to produce a .png, and removes the temporary .dot file. The resulting image shows the optimised plan visually, exactly as required by the assignment.

7. Allowed Data Types

Each column in a DataFrame has a fixed type that cannot be changed once the column is created. The library accepts the following Arrow types:

Logical Type	Description
int32	32-bit signed integer
int64	64-bit signed integer (default for integer literals)
float32	32-bit floating point

Logical Type	Description
float64	64-bit floating point (default for floating literals)
string	string
boolean	true / false

Important rules our code follows:

- Missing values are always represented as Arrow nulls - never as NaN.
- Type promotion: int + float yields float (BinaryExpression converts both operands to double internally for arithmetic).
- Any operation that has a null operand produces a null result. Arithmetic on nulls, string operations on nulls, and comparisons against nulls all propagate null.
- Type incompatibilities (for example adding a string column and a numeric column) are rejected by Arrow and propagate as a runtime error.

8. Using DataFrameLib in Other Projects

To use the library in another C++ project, two things are required:

- link against the compiled DataFrameLib (a static or shared library produced by our CMakeLists.txt),
- include the master header so all the public symbols become visible:

```
#include "dataframelib.h"
using namespace dataframelib;
```

This single header pulls in the full public API: EagerDataFrame, LazyDataFrame, GroupedDataFrame, the Expression hierarchy, and the helper functions read_csv, read_parquet, scan_csv, scan_parquet, from_columns. Apache Arrow is a transitive dependency, so the user must also have Arrow available on their include path - this is exactly how Polars / Arrow C++ already work.

A minimal example that uses both modes:

```
#include "dataframelib.h"
using namespace dataframelib;

int main() {
    // Eager
    auto df = read_csv("employees.csv");
    df.filter(col("age") > 30)
      .select({"name", "salary"})
      .write_csv("out.csv");

    // Lazy
    auto plan = scan_parquet("data.parquet")
        .filter(col("age") > 30)
        .group_by({"dept"})
        .aggregate({{"avg_sal", col("salary").mean()}});

    plan.explain("plan.png");
    auto result = plan.collect();
}
```

9. Input Format and Correct Usage

Inputs to the library come in two forms - data files on disk and code (the operations the user writes).

9.1 Data files

- **CSV:** standard comma-separated files with a header row. Arrow handles quoting and escaping. Pass the file path as a std::string to read_csv / scan_csv.
- **Parquet:** any standard Parquet file. Pass the path to read_parquet / scan_parquet.
- **In-memory:** use from_columns(...) with a map / unordered_map / vector of pairs / initializer_list of (name, arrow::Array).

9.2 Operation chaining

Operations follow a fluent pattern. The same syntax works in both eager and lazy mode - the only difference is the starting function:

```
// EAGER - executes immediately
auto df = read_csv("data.csv")
    .filter(col("price") > 100)
    .with_column("tax", col("price") * 0.18)
    .select({"id", "price", "tax"})
    .sort({"price"}, false)
    .head(20);

// LAZY - builds plan, executes on collect()
auto df = scan_parquet("data.parquet")
    .filter(col("price") > 100)
    .group_by({"category"})
    .aggregate({{"total", col("price").sum()}})
    .collect();
```

9.3 Building Expressions

Expressions are built using:

- **col("name"):** reference a column by name.
- **lit(value):** wrap a constant. Overloaded for int, int64_t, double, bool, std::string, const char*.
- Operators: + - * / %, comparisons == != < <= > >=, boolean & | ~.
- Methods on Expr: .alias(name), .sum(), .mean(), .count(), .min(), .max(), .length(), .contains(s), .starts_with(s), .ends_with(s), .to_lower(), .to_upper(), .is_null(), .is_not_null(), .abs().

10. Exception Handling

Errors in DataFrameLib are reported through standard C++ exceptions - no error codes, no silent failures. The library throws two main types:

- **std::invalid_argument:** for user-facing logical errors - missing column, empty join keys, unknown join type, unknown aggregation function, etc.
- **std::runtime_error:** for I/O and Arrow-level failures - file open failure, CSV reader failure, Parquet read/write failure, builder failure, unknown expression type.

10.1 Where errors are caught

Each public method validates its inputs early and throws a clear message:

```
// select - column not found
if (!col)
    throw std::invalid_argument(
        "select: column not found: " + name);

// join - empty key list
if (on.empty())
```

```
    throw std::invalid_argument(
        "join: 'on' keys must not be empty");

// join - unknown join type
if (how != "inner" && how != "left" &&
    how != "right" && how != "outer")
    throw std::invalid_argument("Invalid join type");

// read_csv - file open failure
if (!file_result.ok())
    throw std::runtime_error("Failed to open file: " + path);
```

10.2 Wrapped exceptions

Some operations (like join()) wrap their entire body in a try/catch and re-throw with extra context, so the user sees exactly which operation failed:

```
try {
    /* join logic */
} catch (const std::exception& e) {
    throw std::runtime_error(
        "Join failed (" + how + "): " + std::string(e.what()));
}
```

10.3 Lazy plan errors

Errors in the lazy path are deferred to collect() time, since no work happens until then. execute() validates that:

- non-leaf nodes have at least one child,
- a join node has a right_child,
- an aggregate node has a group_by child.

```
if (node->children.empty())
    throw std::runtime_error("Invalid plan: missing child node");

if (!node->right_child)
    throw std::runtime_error("Join missing right child");

throw std::runtime_error("aggregate must follow group_by");
```

10.4 Null and Arrow-level errors

Inside expressions, every scalar access uses Arrow's Result<...> API, and we check the .ok() flag before using the value. If a scalar is invalid (null), our helpers (get_bool_value_safe, append_scalar_to_builder) treat it as null and continue rather than crashing - this matches the assignment's rule that any operation involving a null operand must produce a null result.

10.5 Optimiser safety

The optimiser is recursive. To guarantee termination even if a future rule accidentally creates a cycle, optimize() carries a depth counter and bails out at MAX_STEPS = 200 with a warning, returning the partially-optimised plan. This means the worst case for the optimiser is "no further optimisation" - it never crashes the program.

Together, these layers ensure that whether the failure is due to a missing file, a bad column reference, a malformed plan, or a malformed expression, the user receives a meaningful exception with enough information to fix the problem.